Dictionaries

Dictionaries

Initialization

In [ ]:
```
d = {
    'a': '1',
    'b': '2',
    'c': '3',
    'd': '4',
}
print(d)
```

```
{'a': '1', 'b': '2', 'c': '3', 'd': '4'}
```

In [ ]:
```
d = dict(a='1', b='2', c=[3, 3.5, 3.7], d='4')
print(d)
```

```
{'a': '1', 'b': '2', 'c': [3, 3.5, 3.7], 'd': '4'}
```

In [ ]:
```
d = dict([('a', '1'), ('b', '2'), ('c', '3'), ('d', '4')])
print(d)
```

```
{'a': '1', 'b': '2', 'c': '3', 'd': '4'}
```

Properties

Dictionaries are ordered.

In [ ]:
```
d = dict(zip(['a', 'b', 'c', 'd'], ['1', '2', '3', '4']))
print(d)
d = dict(zip(['b', 'c', 'a', 'd'], ['2', '3', '1', '4']))
print(d)
```

```
{'a': '1', 'b': '2', 'c': '3', 'd': '4'}
{'b': '2', 'c': '3', 'a': '1', 'd': '4'}
```

But they cannot be indexed by integers. Instead, they are indexed by keys.

In [ ]:
```
print(d['a'])
```

```
1
```

The keys must be hashable (meaning they must be immutable). Non-unique keys are overwritten.

In [ ]:
```
# this is invalid
d = {
    's': 1
    [1, 2]: 2,  # SyntaxError: invalid syntax
    3: 3
    's': 4      # will overwrite the previous 's'
}
```

In [ ]:
```
d = {
    1: 'a',
    2: 'b',
    0: 'c',
    1.0: 'c',
    1.1: 'd',
    False: 'd',
}
print(d)
```

```
{1: 'c', 2: 'b', 0: 'd', 1.1: 'd'}
```

A dictionary can be copied using the `copy` method.

In [ ]:
```
d1 = { 'a': 1, 'b': 2 }
d2 = d1.copy()
# here changing d1 will not affect d2
d1['a'] = 3
print(d2)
print(id(d1), id(d2))
```

```
{'a': 1, 'b': 2}
127284579202240 127284579200896
```

A dictionary can be created using the fromkeys method.

In [ ]:
```
keyList = ['a', 'b', 'c', 'd']

d = dict.fromkeys(keyList)
print(d)
```

```
{'a': None, 'b': None, 'c': None, 'd': None}
```

In [ ]:
```
keyList = ['a', 'b', 'c', 'd']

d = dict.fromkeys(keyList, 0)
print(d)
```

```
{'a': 0, 'b': 0, 'c': 0, 'd': 0}
```

Keys, values, items

In [ ]:
```
d = {
    'hello': 'there',
    'general': 'kenobi',
    'you': 'are',
    'a': 'bold',
    'one': '!'
}

print(d.get('hello'))
print(d.get('missing_key'))
print(d.get('missing_key', 'default_value'))
```

```
there
None
default_value
```

In [ ]:
```
print(d.items())
```

```
dict_items([('hello', 'there'), ('general', 'kenobi'), ('you', 'are'), ('a', 'bold'), ('one', '!')])
```

In [ ]:
```
print(d.keys())
```

```
dict_keys(['hello', 'general', 'you', 'a', 'one'])
```

In [ ]:
```
print(d.values())
```

```
dict_values(['there', 'kenobi', 'are', 'bold', '!'])
```

In [ ]:
```
for k, v in d.items():
    print(k, v)
```

```
hello there
general kenobi
you are
a bold
one !
```

Modifying a dictionary

In [ ]:
```
d = {
    'hello': 'there',
    'general': 'kenobi',
    'you': 'are',
    'a': 'bold',
    'one': '!'
}
```

In [ ]:
```
d['hello'] = 'hi'
d
```

Out[ ]:
```
{'hello': 'hi', 'general': 'kenobi', 'you': 'are', 'a': 'bold', 'one': '!'}
```

The `dict.pop(key)` method removes the key and returns the value. It doesn't work without an argument.
The `dict.popitem()` method removes the last key-value pair and returns it.
The `dict.clear()` method removes all key-value pairs.

In [ ]:
```
d.pop('hello')
```

Out[ ]:
```
'hi'
```

In [ ]:
```
d
```

Out[ ]:
```
{'general': 'kenobi', 'you': 'are', 'a': 'bold', 'one': '!'}
```

In [ ]:
```
d.pop("random_bullshit_go")
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[3], line 1
----> 1 d.pop("random_bullshit_go")

KeyError: 'random_bullshit_go'
```

To avoid a KeyError in `dict.pop(key)`, you can pass a default value as a second argument as `dict.pop(key, default_value)`.
To avoid a KeyError in `dict[key]`, you can use the `dict.get(key)` method.

In [ ]:
```
d.pop('missing_key', 'default_value')
```

Out[ ]:
```
'default_value'
```

In [ ]:
```
d.setdefault('hello', 'there')
```

Out[ ]:
```
'there'
```

In [ ]:
```
d
```

Out[ ]:
```
{'general': 'kenobi', 'you': 'are', 'a': 'bold', 'one': '!', 'hello': 'there'}
```

In [ ]:
```
d = {
    'hello': 'there',
    'general': 'kenobi',
    'you': 'are',
    'a': 'bold',
    'one': '!'
}
```

In [ ]:
```
d.popitem()
```

Out[ ]:
```
('one', '!')
```

In [ ]:
```
d
```

Out[ ]:
```
{'hello': 'there', 'general': 'kenobi', 'you': 'are', 'a': 'bold'}
```

`dict.update()` essentially merges two dictionaries into the first. It returns None. It acts kind of like the `extend` method for lists, or like union update in set theory.

In [ ]:
```
d = {
    'hello': 'there',
    'general': 'kenobi',
    'you': 'are',
    'a': 'bold',
    'one': '!'
}
```

In [ ]:
```
d.update({'new_key': 'new_value'})
d
```

Out[ ]:
```
{'general': 'kenobi',
 'you': 'are',
 'a': 'bold',
 'one': '!',
 'hello': 'there',
 'new_key': 'new_value'}
```

It also accepts a list of tuples. Like `dict.update([('a', 1), ('b', 2)])`.
Or even a list of lists. Like `dict.update([['a', 1], ['b', 2]])`.
Or even a string of tuples. Like `dict.update(a=1, b=2)`.

In [ ]:
```
d.update(i='dont',know='what', im='doing')
```

In [ ]:
```
'hello' in d
```

Out[ ]:
```
True
```

In [ ]:
```
d
```

Out[ ]:
```
{'general': 'kenobi',
 'you': 'are',
 'a': 'bold',
 'one': '!',
 'hello': 'there',
 'new_key': 'new_value',
 'i': 'dont',
 'know': 'what',
 'im': 'doing'}
```